

SpiceCRM Core Voice Assistant

Technical Design Document

Architecture and implementation design for voice-driven CRM updates

Document item	Value
Target platform	SpiceCRM Core / open-source codebase
Primary use case	Push-to-talk CRM data entry and follow-up creation
Design status	Architecture design for implementation planning
Prepared	September 1, 2026
Key principle	AI proposes; SpiceCRM resolves, validates, confirms, and writes
Revision	1.1 - commit idempotency/concurrency, multi-record failure handling, and per-user rate limiting

Design objective: Make post-meeting CRM entry take seconds while preventing silent or ambiguous data writes. The first release is intentionally not an autonomous always-listening agent.

1. Executive summary

This design adds a lightweight voice-entry capability to the SpiceCRM Core frontend and backend. A user presses a microphone button, speaks a natural-language update, reviews the system's interpretation in an editable confirmation card, and explicitly saves the proposed CRM changes. The language model never receives authority to invent record IDs, execute SQL, or write directly to the CRM.

The recommended V1 architecture separates five responsibilities: browser audio capture, speech-to-text, structured intent extraction, SpiceCRM record resolution, and SpiceCRM-controlled writes. This separation keeps the integration testable and provider-neutral while preserving existing authentication, ACL, search, and activity behavior. The commit path is additionally protected by durable idempotency controls, and the provider-facing interpret endpoint is protected by per-user rate limits and concurrency caps so retries, UI defects, or compromised sessions cannot create duplicate CRM records or unbounded provider spend.

Decision	Recommendation
Primary UI entry point	Persistent microphone in GlobalHeaderTools.
Secondary UI entry point	Context microphone on record-detail views; current record is passed as target context.
Capture	Push-to-talk MediaRecorder; WebM/Opus when supported.
STT	Benchmark OpenAI GPT-Transcribe vs Deepgram Nova-3 using CRM-name accuracy; start with one provider behind an adapter.
Intent extraction	LLM Structured Outputs / JSON Schema in V1; tool-calling agent deferred.
Resolution	SpiceCRM FTS/global search; LLM never supplies authoritative record IDs.
Write policy	No write before confirmation. Server-side commit endpoint performs validated bean writes.
Data retention	Store transcript and AI provenance; raw audio retention configurable and preferably short-lived.

2. User experience and microphone placement

The microphone should be discoverable without competing with the existing module search. The screenshot below shows two placements on the Opportunities list screen. Placement A is the recommended persistent entry point. Placement B illustrates a module-context action; on a record-detail screen, the equivalent button should sit beside the record header so the assistant can inherit the current record ID and avoid name resolution.

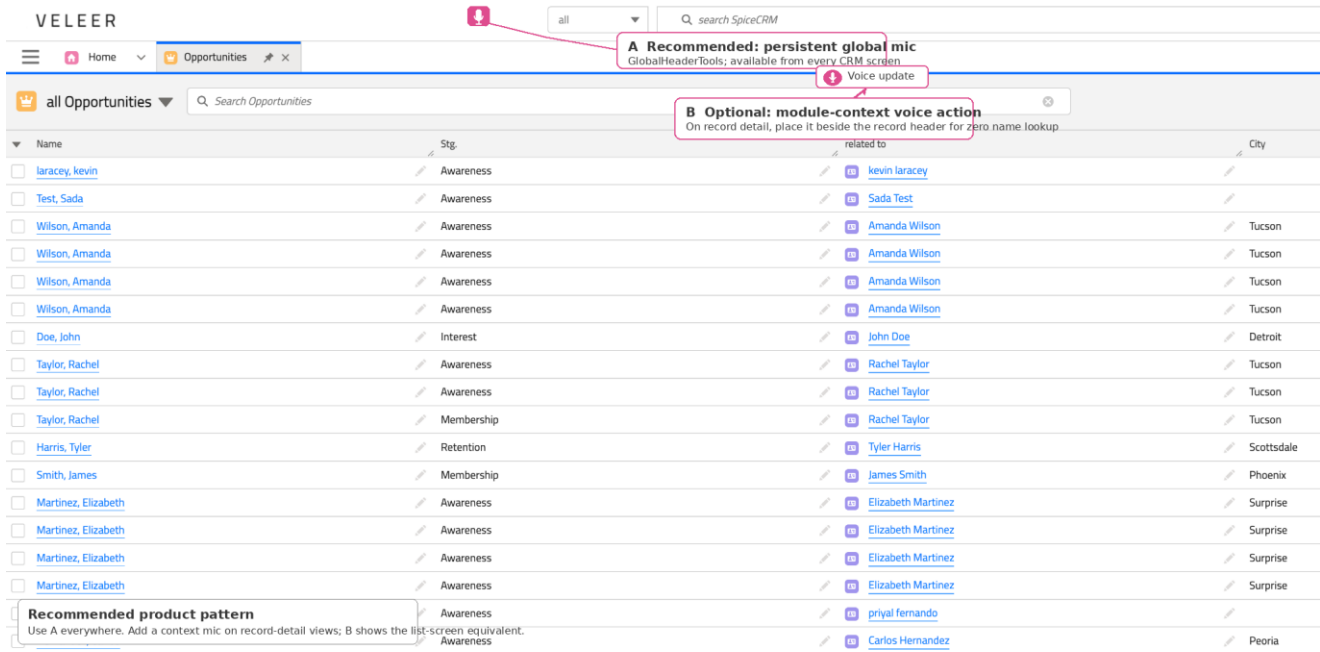


Figure 1. Recommended microphone placements on the supplied SpiceCRM Opportunities screen.

Placement	Purpose	Recommendation
A - Global header	Available from any CRM screen; ideal for commands such as “Find Eric Poe and add a meeting summary.”	Implement in V1 through the GlobalHeaderTools component set. Reserve a compact 32–36 px icon control and preserve global search width responsively.
B - Module/list context	Can scope searches to Opportunities and provides a visible “Voice update” action on the current module.	Optional. Useful if user testing shows the mic icon alone is not discoverable.
Record-detail context	Uses the loaded bean/module as authoritative context; e.g., on Eric Poe, “add yesterday’s meeting” needs no person lookup.	Strongly recommended in V1 or V1.1 because it materially improves reliability.

2.1 Primary interaction

1. User presses the microphone button. The browser requests microphone permission if required and starts recording.
2. UI changes to a visible Listening state with Stop/Cancel controls and an elapsed timer.
3. On stop, the audio is sent to the authenticated Voice Assistant backend endpoint.
4. Backend transcribes the audio, asks the LLM for a schema-constrained proposal, and resolves any named CRM target through SpiceCRM search.
5. Frontend displays an editable confirmation card including matched record, meeting/task/note fields, and transcript disclosure.
6. User presses Save. A second authenticated backend call validates and commits the approved changes through SpiceCRM bean logic.
7. UI reports the created/updated records and offers links to open them.

2.2 Confirmation-card requirements

- Display the resolved CRM record and a Change/Swap control when name resolution was required.
- All dates, times, subjects, summaries, task type, and due dates/windows must be editable before commit.
- Show “Save N updates” rather than a generic Save button so the side effects are obvious.
- Expose the raw transcript in a collapsible region for troubleshooting without making it the primary UI.
- Never hide ambiguity. Multiple plausible records require an explicit picker.
- Fuzzy dates such as “week of August 25” should remain semantically visible until a configured business rule maps them to a concrete due date.

3. Scope

3.1 V1 supported actions

Action	V1 behavior
Find Contact / Prospect / Opportunity	Resolve by FTS and present matched record.
Create Meeting	Create a held/past meeting with date/time, description, assigned user, and normalized relationship.
Create Note	Create note attached to the selected target.
Create Task / follow-up	Create a task using configured mapping for exact or fuzzy due date.
Edit proposed fields	Allowed before commit in confirmation card.

3.2 Explicitly out of scope for V1

- Always-listening or wake-word behavior.
- Autonomous writes without user confirmation.
- Deleting CRM records.
- Sending email or SMS.
- Arbitrary SQL, generic REST execution, or direct LLM access to database credentials.
- Opportunity-stage changes until the confirmation and audit model has been proven.
- Multi-turn realtime voice conversation; add only if ambiguity-resolution usage justifies it.

4. Logical architecture

Layer	Component	Responsibility
Frontend	GlobalHeaderVoiceAssistant	Persistent microphone, recording state, upload, proposal display.
Frontend	ObjectVoiceAssistant	Same flow but seeded with current module + record ID.
Frontend	VoiceAssistantService	HTTP coordination, proposal state, errors, retry/cancel.
Backend	VoiceAssistant API extension	Authenticated routes for interpret and commit.
Backend	TranscriptionProvider adapter	OpenAI or Deepgram STT implementation; keyword/context hints.
Backend	IntentInterpreter adapter	LLM Structured Outputs against strict CRM command schema.
Backend	RecordResolver	SpiceCRM FTS search and ambiguity scoring.

Backend	ActivityWriteAdapter	Normalizes Contacts/Prospects/Opportunities and creates Meetings/Tasks/Notes using SpiceCRM bean logic.
Backend	Audit/telemetry	Transaction ID, transcript, model/provider, latency, error and outcome logging.

Boundary rule: The model may output human-understandable names and proposed actions, but only SpiceCRM may resolve authoritative bean IDs and commit data.

4.1 Sequence

```

Browser mic
-> MediaRecorder
-> POST /voiceassistant/interpret
-> STT provider
-> transcript
-> LLM Structured Output
-> RecordResolver / SpiceCRM FTS
-> proposed transaction
-> confirmation card
-> POST /voiceassistant/commit
-> ACL + optimistic validation
-> ActivityWriteAdapter
-> Meetings / Tasks / Notes
-> audit + response

```

5. SpiceCRM Core frontend integration

SpiceCRM release documentation identifies GlobalHeaderTools as the component set used for global-header tools and documents GlobalHeaderModuleShortcut as an example. The Voice Assistant should follow that configurable component-set mechanism rather than hard-forking a header template. The architect should verify the exact metadata/component registration syntax against the deployed SpiceCRM tag before coding.

5.1 Proposed frontend components

```

src/.../globalheadervoiceassistant/
  globalheadervoiceassistant.ts
  globalheadervoiceassistant.html

```

```

src/.../objectvoiceassistant/
  objectvoiceassistant.ts
  objectvoiceassistant.html

```

```

src/services/voiceassistant.service.ts

```

Exact custom-extension paths should follow the Core custom/extensions conventions in the deployed release; avoid editing a core component if the same behavior can be registered as custom metadata.

5.2 MediaRecorder behavior

- Use navigator.mediaDevices.getUserMedia({audio:true}) only after an explicit user click.
- Prefer MediaRecorder output in WebM/Opus; include MIME type in the upload metadata.
- Set a V1 maximum recording duration, e.g., 90 seconds, with a visible countdown/elapsed time.
- Stop microphone tracks immediately on Stop, Cancel, navigation, or component destruction.
- Do not store audio in browser persistent storage.
- Surface microphone permission failure with browser-specific remediation text.

5.3 State model

State	UI
idle	Mic button; tooltip “Voice update”.
requesting_permission	Spinner; cancel disabled only while permission prompt is active.
recording	Red/listening state, timer, Stop and Cancel.
uploading	Progress/processing label; allow cancel if request supports abort.
proposal	Editable confirmation card.
committing	Disable duplicate Save; show progress.
success	Created/updated record links and concise success message.
error	Actionable error and Retry; preserve transcript/proposal when safe.

6. SpiceCRM Core backend integration

Implement an authenticated custom REST extension using SpiceCRM RESTManager patterns from the deployed Core version. Keep OpenAI/Deepgram credentials in server-side configuration or a secret manager. The frontend must never receive provider API keys.

6.1 Proposed routes

Method / route	Purpose
POST /voiceassistant/interpret	Accept audio or text, transcribe if needed, extract intent, resolve candidate records, and return a proposed transaction. No CRM writes.
POST /voiceassistant/commit	Accept the proposal ID plus user-approved edits; revalidate current user, record IDs, ACLs, dates, and proposal expiry; write all changes.
GET /voiceassistant/config	Optional: return enabled providers, max recording duration, feature flags, and retention policy. No secrets.

The route strings above are logical internal route names. The external URL prefix should follow the deployed SpiceCRM API/proxy configuration rather than being hard-coded as /api/v4 in custom registration.

6.2 Suggested backend class boundaries

VoiceAssistantController

|- interpret()

|- commit()

TranscriptionProviderInterface

|- OpenAITranscriptionProvider

`- DeepgramTranscriptionProvider (optional benchmark/alternate)

IntentInterpreter

RecordResolver

ActivityWriteAdapter

VoiceAssistantAuditService

7. Speech-to-text design

Proper-name accuracy is a primary success metric because a transcription error in a person name can prevent record resolution. OpenAI GPT-Transcribe currently supports context and keyword hints; Deepgram Nova-3 provides keyterm prompting. The design should hide provider-specific behavior behind an adapter and make the initial provider decision using a representative CRM-name benchmark.

7.1 CRM-derived vocabulary

- For global commands, build a bounded hint set from records relevant to the current user: recently viewed, recently contacted, assigned records, open opportunities, and favorites.
- Prefer full names and organization names over surnames alone.
- Do not send an unbounded CRM directory to the STT provider; enforce provider limits and privacy rules.
- For record-context dictation, do not depend on the spoken record name at all; pass the current bean ID as trusted context.

7.2 Provider acceptance benchmark

Metric	Suggested gate
Exact person-name transcription	>= 97% on internal test set with hints enabled.
Date/time transcription	>= 98% correct lexical transcription; semantic normalization tested separately.
Median transcription latency	Target < 2.5 seconds for 15–30 second recording, measured in production-like region.
Cost	Track per minute but do not optimize before accuracy and latency.

8. Intent extraction and structured output

V1 should use schema-constrained Structured Outputs rather than an autonomous tool-calling loop. The LLM receives the transcript, current date, user timezone, optional trusted record context, and a concise set of permitted action types. It returns JSON matching the application schema or an explicit needs_clarification result.

8.1 Example proposal schema

```
{
  "target": {
    "context_record_id": "uuid-or-null",
    "context_module": "Prospects|Contacts|Opportunities|null",
    "lookup_query": "Eric Poe|null"
  },
  "actions": [
    {
      "type": "create_meeting",
      "occurred_at_local": "2026-08-18T14:00:00",
      "duration_minutes": null,
      "summary": "Great meeting. Eric offered to edit our decks to improve them."
    },
    {
      "type": "create_followup",
      "kind": "call",
      "subject": "Schedule follow-up call",
      "due": {"kind": "week_of", "date": "2026-08-25"}
    }
  ],
  "needs_clarification": []
}
```

8.2 Interpreter rules

- Current record context is authoritative and must not be replaced based on model inference unless user explicitly names a different target.
- If a value is absent, return null or a clarification request; never fabricate a duration, email, account, or record ID.
- Resolve relative dates using server-provided current date and the user timezone.
- Keep fuzzy temporal intent as a structured semantic value (week_of, next_week, end_of_month) until application policy maps it to a concrete date.
- Keep dictated summary meaning faithful; light punctuation cleanup is allowed, but do not invent factual content.

9. Record resolution

Named targets must be resolved with SpiceCRM search/FTS, not by the LLM. The RecordResolver receives the target module set and lookup_query and returns ranked candidates with only the fields required for disambiguation.

Outcome	Behavior
Trusted record context supplied	Use current record ID after ACL/read validation; no name search.
One high-confidence candidate	Return as proposed target and show it on confirmation card.
Multiple plausible candidates	Return candidate picker; do not infer silently.
No candidate	Return unresolved state; allow retry/edit or cancel. Creating a new person is out of V1 scope.

9.1 Candidate scoring

- Exact normalized full-name match should dominate fuzzy matches.
- Use module context and user visibility/assignment as secondary signals, not as a substitute for a name match.
- Never expose candidates the user lacks read access to.
- Log resolution score and candidate count for quality analysis, but do not rely on an LLM-generated “confidence score.”

10. CRM write adapter and activity relationships

A dedicated ActivityWriteAdapter should hide SpiceCRM-specific relationship differences from the AI layer. Meetings, Tasks, and Notes can have different relationship semantics for Contacts, Prospects, and Opportunities. The adapter decides whether to populate polymorphic parent fields and/or module-specific relationship links so that the resulting activity appears correctly in the target record’s activity stream.

10.1 Write principles

- Use SpiceCRM bean/domain logic and existing ACL checks; do not write SQL directly.
- Set assigned_user_id from authenticated user or an explicit approved assignment policy, not from the model.
- Convert local meeting date/time to the backend-required timezone/UTC representation consistently.
- For past meetings, set status according to your CRM convention (commonly Held) only if that convention is configured and tested.
- Define a deterministic business rule for fuzzy follow-up windows. Example: “week of 8/25” remains a window in the proposal; on commit the user may choose a date or the organization may configure Monday/noon as the default.
- Verify each relationship mapping by checking the activity stream and related lists in the exact Core build in use.

10.2 Transaction semantics

Commit MUST be idempotent and concurrency-safe. Every proposal receives a proposal_id, and every user Save attempt receives an idempotency_key that is reused unchanged for network retries of that same approved payload. The server persists the idempotency state before any CRM write and enforces a unique key per authenticated user. A duplicate request with the same key and same payload returns the original result; the same key with a different payload is rejected. Multi-

record commits must either execute inside a verified shared database transaction or use a defined compensating-write strategy; "best effort" partial writes are not an acceptable silent outcome.

10.2.1 Idempotency record

- Create a server-side `voice_assistant_commits` (or equivalent) persistence record keyed by (`authenticated_user_id`, `idempotency_key`) with a database UNIQUE constraint; client-side Save disabling is usability only, not the duplicate-prevention mechanism.
- Persist at minimum: `user_id`, `idempotency_key`, `proposal_id`, `canonical_request_hash`, `transaction_id`, `status` (`processing`|`committed`|`failed`|`partial_failure`), `response_payload/reference`, `created_at`, `updated_at`, and `expires_at`.
- Compute `canonical_request_hash` from the approved target and approved_actions after normalization (for example canonical JSON + SHA-256). The idempotency key itself is excluded from the hash.
- Recommended V1 idempotency lookup TTL: 24 hours, and never shorter than the proposal-expiry period plus the expected retry window. Audit records may be retained longer under the organization's audit policy.

10.2.2 Concurrent duplicate behavior

- First request: atomically INSERT the idempotency row with `status=processing` before any Meeting/Task/Note write. The UNIQUE constraint is the race arbiter across PHP workers and application nodes.
- Same user + same key + same hash + `status=committed`: return HTTP 200 with the previously stored commit response and created record IDs; do not execute bean saves again.
- Same user + same key + same hash + `status=processing`: return HTTP 202 `commit_in_progress` with the existing `transaction_id` and a short Retry-After value, or block briefly and return the completed cached result. The implementation plan must choose one behavior consistently.
- Same user + same key + different request hash: return HTTP 409 `idempotency_key_reuse`. The client must generate a new key when the user materially changes the approved payload and initiates a new commit attempt.
- A failed or partial commit must never be blindly replayed. The persisted status determines whether a retry may safely resume, whether compensation/repair is required, or whether the user must generate a fresh proposal.

10.2.3 Multi-record atomicity / compensation

- Preflight all actions before the first write: proposal expiry, target existence, ACLs, field validation, date conversion, relationship mapping, and required configuration. This prevents avoidable partial commits.
- Preferred implementation: wrap all bean writes for one approved proposal in one database transaction only if the deployed SpiceCRM bean/database layer supports it and all relevant logic hooks/side effects have been verified to be rollback-safe.
- Fallback implementation: use a commit coordinator/saga. Create records in a deterministic order, record every created bean ID under `transaction_id`, and on failure run explicit compensation for records created by that transaction when policy and hooks make compensation safe.
- If compensation cannot restore a clean state, persist `status=partial_failure`, return the exact successful/failed record IDs, write a high-severity audit event, and surface an administrator repair action. The UI must not report the overall operation as successful.

11. API contracts

11.1 Interpret request

POST /voiceassistant/interpret
Content-Type: multipart/form-data

audio: <webm/opus blob> # optional if text supplied
text: <typed command> # optional alternate input
context_module: Opportunities # optional
context_record_id: <uuid> # optional trusted UI context
timezone: America/New_York
client_request_id: <uuid>

11.2 Interpret response

```
{
  "proposal_id": "uuid",
  "transcript": "...",
  "target": {
    "status": "resolved|ambiguous|unresolved|context",
    "record": {"module": "Prospects", "id": "...", "label": "Eric Poe"},
    "candidates": []
  },
  "actions": [...],
  "warnings": [],
  "expires_at": "2026-09-01T16:00:00Z"
}
```

11.3 Commit request

```
POST /voiceassistant/commit
{
  "proposal_id": "uuid",
  "idempotency_key": "uuid",
  "target_record": {"module": "Prospects", "id": "..."},
  "approved_actions": [...edited values from confirmation card...]
}
```

11.4 Commit response

```
{
  "status": "committed",
  "records": [
    {"module": "Meetings", "id": "...", "label": "Meeting with Eric Poe"},
    {"module": "Tasks", "id": "...", "label": "Schedule follow-up call"}
  ],
  "transaction_id": "..."
}
```

11.5 Retry, idempotency, and throttling responses

200 committed/replayed Existing successful commit result; no duplicate writes.
 202 commit_in_progress Same idempotency key is currently executing.
 409 idempotency_key_reuse Key reused with a different approved payload.
 409 partial_failure Commit requires repair/compensation; do not blind-retry.
 429 rate_limited Per-user/deployment limit exceeded; include Retry-After.

12. Security, privacy, and authorization

Control	Requirement
Authentication	Both endpoints require the normal SpiceCRM authenticated session/token.
Authorization	Re-check read access on target and create/edit access on each module at commit time.
Provider keys	Server side only; preferably injected through environment/secret manager.
Prompt injection	Treat dictated text as user data. The system instruction/schema defines permitted actions; transcript cannot expand permissions.
Data minimization	Send only transcript, date/time context, bounded STT hints, and minimal metadata required by the provider.

Audio retention	Default to delete after successful transcription unless support policy requires temporary retention.
Transcript retention	Store only under an explicit organizational policy; protect with the same CRM access controls where exposed.
Audit	Record authenticated user, provider/model, proposal, edits, final writes, and transaction ID.
Per-user rate limiting	Apply authenticated-user limits before any STT/LLM provider call. Recommended V1 default: 6 interpret requests/minute, 100/hour, and no more than 2 concurrent interpret requests per user. Make limits configurable.
Commit throttling	Recommended V1 default: 12 new commit attempts/minute per user. Exact idempotent replays may return the cached result without consuming another commit execution slot.
Deployment/organization guard	Also support a deployment- or organization-wide cap to protect provider budget and shared capacity (recommended starting point: 60 interpret requests/minute, adjusted for active seat count).
Rate-limit response	Return HTTP 429 with Retry-After and a machine-readable <code>retry_after_seconds</code> . Enforce in a shared store so multiple PHP workers/nodes see the same counters; Redis is preferred when available, database-backed counters are an acceptable fallback.

12.1 Per-user rate limiting and abuse control

- Rate-limit key: authenticated SpiceCRM user ID, not IP address. IP-based controls may be additive but must not replace identity-based limits because many users may share a corporate NAT/proxy.
- Interpret processing order: authenticate -> validate request size/duration -> short-lived `client_request_id` replay lookup (if implemented) -> per-user/deployment limiter -> STT/LLM call. A rate-limited request must not invoke a paid provider.
- Recommended V1 per-user defaults: 6 /voiceassistant/interpret requests per rolling minute, 100 per rolling hour, and max 2 in-flight interpret requests. These are intentionally far above normal human push-to-talk usage while still bounding automated abuse.
- Recommended V1 /voiceassistant/commit default: 12 new commit attempts per rolling minute per user. Before applying the new-attempt limit, check whether the supplied idempotency key already maps to a completed result; an exact replay can be answered from the idempotency store without re-executing the commit.
- Use a shared token-bucket or sliding-window implementation. Do not use process-local PHP memory for authoritative limits in a multi-worker deployment. Preferred store: Redis; fallback: a small database-backed counter table with atomic updates and expiry.
- Return HTTP 429 with Retry-After. The frontend displays a human message such as “Too many voice requests. Try again in 12 seconds,” preserves any local transcript/proposal when safe, and must not automatically retry in a tight loop.
- Make limits configurable through deployment settings/feature flags, for example `voice.rate_limit.interpret_per_minute`, `voice.rate_limit.interpret_per_hour`, `voice.rate_limit.interpret_concurrency`, `voice.rate_limit.commit_per_minute`, and `voice.rate_limit.deployment_per_minute`.
- Log rate-limit events with `authenticated_user_id`, endpoint, limit bucket, retry-after interval, and whether a provider call was prevented. Do not log audio simply because a request was throttled.

13. Audit and observability

Voice features fail in ways that ordinary CRUD does not: transcription, temporal interpretation, name resolution, and model extraction can each be wrong. Instrument them separately so the team can identify the failing stage.

Field / metric	Purpose
----------------	---------

voice_request_id / transaction_id	End-to-end tracing.
authenticated_user_id	Who initiated and committed.
input_mode	voice or text.
stt_provider / model	Provider quality comparison.
transcript	Debugging and audit subject to retention policy.
intent_model / schema_version	Reproducibility across model/schema upgrades.
resolver_candidates / selected_id	Diagnose name failures.
user_edits_before_commit	Best product-quality metric: what the AI got wrong.
latency per stage	STT, LLM, resolution, commit.
created record IDs	Audit and support.
idempotency_key / request_hash / commit_status	Diagnose retries, concurrency races, replay behavior, and partial failures.
rate_limit_bucket / retry_after / provider_call_prevented	Abuse/cost telemetry and verification that throttled requests do not reach paid providers.

13.1 Recommended custom metadata

If you choose to put provenance directly on created activity records, use custom fields such as `ai_source`, `ai_transcript_ref` (or `transcript` field if appropriate), `ai_model`, `ai_schema_version`, `ai_processed_at`, and `ai_transaction_id`. Avoid storing raw provider payloads on normal CRM records.

14. Error handling

Failure	User experience / system behavior
Mic permission denied	Explain how to enable microphone; typed-command fallback remains available.
Unsupported recording format	Fallback MediaRecorder type or block with browser compatibility message.
STT timeout/error	Retry without losing local recording when possible; do not proceed to LLM with empty transcript.
Low-quality/empty transcript	Ask user to retry; never guess.
LLM schema failure	Retry once with same schema/model; then fail safely and log.
Ambiguous record	Require picker.
ACL changes between interpret and commit	Reject commit and refresh proposal/permissions.
Duplicate commit/retry	Durable idempotency lookup returns the previous successful result. Concurrent same-key requests must not execute duplicate bean writes.

Partial write failure	Rollback inside a verified database transaction, or execute the defined compensation flow. Persist partial_failure if cleanup is incomplete; never silently retry.
Per-user rate limit exceeded	Return 429 + Retry-After before provider invocation; preserve safe UI state and do not auto-retry aggressively.

15. Testing and acceptance criteria

15.1 Functional test matrix

- Global command with exact unique person name.
- Global command where STT misspells a name but FTS can still offer plausible candidates.
- Two Contacts/Prospects with same name; picker required.
- Record-context command with no spoken name.
- Past meeting with date and time.
- Relative date (“yesterday at 2”).
- Fuzzy follow-up (“week of 8/25”).
- Single utterance creating meeting + task.
- User edits summary/date before Save.
- User cancels proposal: no writes, no orphan records.
- User lacks Tasks create permission.
- Network retry after commit: no duplicates.
- Two concurrent commit requests with the same user/idempotency key and identical payload: exactly one Meeting/Task set is created; second request returns in-progress or cached result.
- Same idempotency key reused with a different approved payload: HTTP 409 and zero additional writes.
- Network timeout after server commit followed by client retry with the same key: original record IDs are returned; no duplicates.
- Multi-record commit fails after the first bean save: verified transaction rollback or compensation path leaves no unreported orphan activity; partial_failure is surfaced if cleanup cannot complete.
- Per-user interpret limit exceeded: 429 is returned, Retry-After is present, and STT/LLM provider invocation count does not increase.
- One user hits the per-user limit while a second user remains below it: the second user can continue normally.
- Rate-limit counters remain correct across two PHP workers/application nodes (shared limiter store).

15.2 Product acceptance targets

Metric	Initial target
Successful end-to-end completion	>= 95% on curated internal utterance set.
Incorrect silent target selection	0%; ambiguity must be surfaced.
Commands requiring any correction	Track baseline; target < 10% after tuning.
Median time from Stop to proposal	Target < 5 seconds; optimize after baseline measurement.
Duplicate records caused by retries	0.
Unauthorized write tests	0 successful writes.
Concurrent duplicate commit executions	0 duplicate bean writes for same user + idempotency key.
Rate-limited interpret provider calls	0 paid STT/LLM calls after a request has been rejected by the rate limiter.

Rate-limit isolation	Per-user throttling must not throttle unrelated users unless the deployment/organization cap is also reached.
----------------------	---

16. Packaging, deployment, and upgrade strategy

The frontend is a compiled Angular application, so custom components require a build/deployment step. Keep the Voice Assistant in the Core custom/extensions area and metadata configuration whenever possible rather than editing shipped Core files. The implementation plan should document every touched path and whether an upstream update can overwrite it.

- Pin the SpiceCRM release/tag used for development and test all extension points against that exact version.
- Maintain the voice feature in its own source directory/package and CI build job.
- Use feature flags/settings: `voice.enabled`, `voice.global_header_enabled`, `voice.record_context_enabled`, `voice.provider`, `voice.audio_retention_hours`, `voice.idempotency_ttl_hours`, `voice.rate_limit.interpret_per_minute`, `voice.rate_limit.interpret_per_hour`, `voice.rate_limit.interpret_concurrency`, `voice.rate_limit.commit_per_minute`, and `voice.rate_limit.deployment_per_minute`.
- Deploy to a non-production environment with representative FTS data and real user ACL profiles.
- After every SpiceCRM upgrade, run a smoke suite covering component registration, FTS resolution, ACLs, and Meetings/Tasks relationship rendering.

17. Recommended implementation phases

Phase	Deliverable	Exit criterion
0 - Technical spike	Standalone or minimal page: MediaRecorder -> STT -> transcript; provider benchmark on CRM names.	Choose STT provider based on measured name accuracy/latency.
1 - Record-context MVP	Mic on Prospect/Contact/Opportunity detail, Structured Output, confirmation card, Meeting/Task/Note commit.	Users can reliably log a meeting + follow-up without speaking the person name.
2 - Global mic	GlobalHeaderTools component, FTS name resolution, ambiguity picker.	Global Eric-Poe-style command works end-to-end.
3 - Hardening	Audit dashboards, durable idempotency store, concurrent-retry tests, atomic/compensating commit behavior, per-user/deployment rate limiting, retention policy, and load tests.	Production readiness/security signoff, including duplicate-write race test and throttling/cost-guard verification.
4 - Expansion	Opportunity stage changes, richer tasks, typed command, optional multi-turn/realtime voice.	Only after V1 usage and correction data justify scope.

18. Decisions the architect should close before implementation

1. Which exact SpiceCRM Core release/tag is deployed, and what are the supported custom frontend/backend extension paths in that tag?
2. Should V1 ship record-context first, or global mic and record-context together?
3. Which modules count as “person” targets: Contacts, Prospects, Leads if present, and/or Opportunities?
4. How should a Meeting be related for Contacts vs Prospects vs Opportunities so it appears correctly in the current activity stream and reporting?
5. How should fuzzy due windows map to a single Task `date_due`, if a single date is required?
6. What audio and transcript retention policy is acceptable for the organization?
7. Which STT provider wins the internal proper-name benchmark?
8. Which LLM/model is approved for Structured Outputs, and what schema-versioning policy will be used?
9. What is the maximum voice command duration and expected peak concurrency?

10. What telemetry may be retained in production for quality analysis?
11. Will multi-record commits use a verified shared database transaction, or a commit-coordinator/saga with compensating writes? Which SpiceCRM logic hooks/side effects must be considered?
12. Where will idempotency records be stored, what is the final TTL (recommended V1: 24 hours), and what exact response is returned for a concurrent same-key request?
13. What per-user limits are approved for /voiceassistant/interpret and /voiceassistant/commit, and what deployment/organization cap protects shared provider budget?
14. Which shared limiter store will be used (Redis preferred; database fallback), and how will rate-limit behavior be tested across multiple PHP workers/nodes?

19. Worked example

Spoken command:

"Find prospect record for Eric Poe, and add a meeting summary for meeting from 8/18 at 2pm and say great meeting, Eric offered to edit our decks to improve them. Next step is to schedule a follow up call for week of 8/25."

19.1 Expected interpretation

Target lookup: Prospects / "Eric Poe"
 Action 1: create meeting
 local datetime: 2026-08-18 14:00
 summary: Great meeting. Eric offered to edit our decks to improve them.
 Action 2: create follow-up
 kind: call
 subject: Schedule follow-up call
 due semantics: week of 2026-08-25

19.2 Confirmation behavior

If one Eric Poe is found, show that record in the card. If multiple records are plausible, pause and require the user to choose. The due window is displayed as "Week of Aug 25" and remains editable. Only after Save does the server create the Meeting and Task and return their IDs.

Revision 1.1 note

This revision incorporates review feedback on commit idempotency/concurrency and abuse controls. It replaces the earlier one-sentence idempotency requirement with a durable server-side state model and adds identity-based rate limiting before paid STT/LLM calls.

20. Source notes and implementation references

These references support the architectural assumptions in this document. The architect should still verify them against the exact SpiceCRM tag deployed because extension points can change between releases.

[1] SpiceCRM release notes - GlobalHeaderTools / GlobalHeaderModuleShortcut and extension architecture:

<https://www.spicecrm.com/the-releases/>

[2] SpiceCRM public repository - current source tree and Core architecture: <https://github.com/spicecrm/spicecrm>

[3] OpenAI GPT-Transcribe - speech-to-text, context/keyword hints, current pricing:

<https://developers.openai.com/api/docs/models/gpt-transcribe>

[4] OpenAI Responses API - Structured Outputs / JSON Schema support: <https://developers.openai.com/api/reference/>

[5] Deepgram Keyterm Prompting - proper-name/domain term prompting (optional alternate STT):

<https://developers.deepgram.com/docs/keyterm>

Implementation handoff: This document is intended to let the architect turn the design into a release-specific implementation plan with file paths, class names, sequence estimates, and test cases. Where the document uses logical

endpoint or component names, the implementation plan should bind them to the exact conventions of the deployed SpiceCRM Core release.